Développement Web Avancé

Framework Laravel 11

Chapitre 3 : Vues et Blade

Enseignant
M. JARRAY Mondher

Classe
MP1ILC

Semestre
S1

Durée
1h30

Objectifs du Chapitre

- ✓ Comprendre le rôle des vues dans l'architecture MVC
- ✓ Maîtriser la syntaxe du moteur de templates Blade
- ✓ Créer des layouts et des composants réutilisables
- ✓ Passer des données du contrôleur à la vue
- ✓ Utiliser les directives de contrôle (if, foreach, etc.)
- ✓ Organiser les vues pour le projet TaskFlow

Plan du Chapitre

1. **Introduction aux Vues**
   - Rôle des vues
   - Créer et afficher une vue
2. **Le Moteur Blade**
   - Affichage de données
   - Directives de contrôle
3. **Layouts et Héritage**
   - Créer un layout principal
   - Sections et yield
4. **Composants Blade**
5. **Inclusion de Sous-vues**

1. Introduction aux Vues

La couche présentation de l'application

Définition

Une **vue** est un fichier qui contient le code HTML de votre application. Elle est responsable de l'affichage des données à l'utilisateur.

Emplacement des vues

```
resources/
    views/
        welcome.blade.php       # Page d'accueil par défaut
        layouts/
            app.blade.php       # Layout principal
        projets/
            index.blade.php     # Liste des projets
            show.blade.php      # Détail d'un projet
            create.blade.php    # Formulaire création
        components/
            alert.blade.php     # Composant réutilisable
```

Convention de nommage

- Extension : `.blade.php`

Créer et Afficher une Vue

Du contrôleur à l'affichage

Étape 1 : Créer le fichier de vue

Créez `resources/views/projets/index.blade.php`

```html
<!DOCTYPE html>
<html>
<head>
    <title>Liste des Projets</title>
</head>
<body>
    <h1>Mes Projets</h1>
    <p>Bienvenue sur TaskFlow !</p>
</body>
</html>
```

Étape 2 : Retourner la vue depuis le contrôleur

```php
// Dans ProjetController.php
public function index()
{
    // 'projets.index' = resources/views/projets/index.blade.php
    return view('projets.index');
}
```

Notation pointée

Passer des Données à la Vue

Du contrôleur vers l'affichage

Méthode 1 : Tableau associatif

```php
public function index()
{
    $projets = ['Projet A', 'Projet B', 'Projet C'];
    $titre = 'Liste des projets';

    return view('projets.index', [
        'projets' => $projets,
        'titre' => $titre
    ]);
}
```

Méthode 2 : Fonction compact()

```php
public function index()
{
    $projets = ['Projet A', 'Projet B'];
    $titre = 'Liste des projets';

    // compact() crée un tableau à partir des noms de variables
    return view('projets.index', compact('projets', 'titre'));
}
```

Méthode 3 : Chaînage with()

2. Le Moteur Blade

Afficher des données

Qu'est-ce que Blade ?

Blade est le moteur de templates de Laravel. Il permet d'écrire du PHP de manière plus simple et sécurisée dans les vues.

Afficher une variable

Syntaxe Blade

```blade
{{-- Échappé (sécurisé) --}}
<h1>{{ $titre }}</h1>
<p>{{ $description }}</p>

{{-- Non échappé (HTML brut) --}}
{!! $contenuHtml !!}
```

Équivalent PHP

```php
// Échappé
<h1><?= htmlspecialchars($titre) ?></h1>

// Non échappé
<?= $contenuHtml ?>
```

Directives Conditionnelles

@if, @else, @elseif

Structure if / else

BLADE

```
@if($projets->count() > 0)
    <p>Vous avez {{ $projets->count() }} projet(s)</p>
@else
    <p>Aucun projet trouvé</p>
@endif
```

Structure complète

BLADE

```
@if($statut === 'termine')
    <span class="badge-success">Terminé</span>
@elseif($statut === 'en_cours')
    <span class="badge-warning">En cours</span>
@else
    <span class="badge-secondary">En attente</span>
@endif
```

Directives utiles

BLADE

```
@isset($projet)
    {{-- S'affiche si $projet existe et n'est pas null --}}
@endisset

@empty($projets)
```

Directives de Boucle

@foreach, @for, @while

@foreach - La plus utilisée

BLADE

```
<ul>
@foreach($projets as $projet)
    <li>{{ $projet->nom }}</li>
@endforeach
</ul>
```

@forelse - Avec gestion du cas vide

BLADE

```
@forelse($projets as $projet)
    <div class="card">
        <h3>{{ $projet->nom }}</h3>
        <p>{{ $projet->description }}</p>
    </div>
@empty
    <p>Aucun projet à afficher</p>
@endforelse
```

Variable $loop

Dans une boucle, vous avez accès à la variable `$loop` :

BLADE

```
@foreach($projets as $projet)
    <p>
```

Propriétés de $loop

Informations sur l'itération en cours

Propriété	Description	Exemple
`$loop->index`	Index actuel (commence à 0)	0, 1, 2...
`$loop->iteration`	Numéro d'itération (commence à 1)	1, 2, 3...
`$loop->remaining`	Itérations restantes	4, 3, 2, 1, 0
`$loop->count`	Nombre total d'éléments	5
`$loop->first`	Est-ce le premier élément ?	true/false
`$loop->last`	Est-ce le dernier élément ?	true/false
`$loop->even`	Itération paire ?	true/false
`$loop->odd`	Itération impaire ?	true/false
`$loop->parent`	Accès à la boucle parente	Boucles imbriquées

Exemple : Alterner les couleurs

BLADE

```
@foreach($projets as $projet)
    <tr class="{{ $loop->even ? 'bg-gray' : 'bg-white' }}">
        <td>{{ $projet->nom }}</td>
    </tr>
@endforeach
```

3. Layouts et Héritage

Éviter la répétition de code

Le problème

Sans layout, vous devez répéter le HTML de base dans chaque page :

page1.blade.php

```html
<!DOCTYPE html>
<html>
<head>
  <title>Page 1</title>
  <link href="style.css"...>
</head>
<body>
  <nav>...</nav>
  <!-- Contenu page 1 -->
  <footer>...</footer>
</body>
</html>
```

page2.blade.php

```html
<!DOCTYPE html>
<html>
<head>
```

Créer un Layout Principal

resources/views/layouts/app.blade.php

BLADE

```
<!DOCTYPE html>
<html lang="fr">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>@yield('title', 'TaskFlow')</title>
    <link href="/css/app.css" rel="stylesheet">
</head>
<body>
    <!-- Navigation -->
    <nav class="navbar">
        <a href="/">TaskFlow</a>
        <a href="/projets">Projets</a>
        <a href="/taches">Tâches</a>
    </nav>

    <!-- Contenu principal (injecté ici) -->
    <main class="container">
        @yield('content')
    </main>

    <!-- Pied de page -->
    <footer>
        © {{ date('Y') }} TaskFlow - ISIM Médenine
    </footer>

    @yield('scripts')
</body>
```

Utiliser le Layout

@extends et @section

Page qui hérite du layout

Fichier : `resources/views/projets/index.blade.php`

BLADE

```blade
{{-- Hériter du layout --}}
@extends('layouts.app')

{{-- Définir le titre --}}
@section('title', 'Liste des Projets')

{{-- Définir le contenu principal --}}
@section('content')
    <h1>Mes Projets</h1>

    @forelse($projets as $projet)
        <div class="card">
            <h2>{{ $projet->nom }}</h2>
            <p>{{ $projet->description }}</p>
            <a href="/projets/{{ $projet->id }}">Voir détails</a>
        </div>
    @empty
        <p>Aucun projet trouvé.</p>
    @endforelse
@endsection
```

Comment ça marche

4. Composants Blade

Éléments réutilisables

Les composants permettent de créer des éléments d'interface réutilisables.

Créer un composant Alert

```
$ php artisan make:component Alert
```

Fichier créé : `resources/views/components/alert.blade.php`

```blade
{{-- Composant Alert --}}
<div class="alert alert-{{ $type ?? 'info' }}">
    @if(isset($title))
        <strong>{{ $title }}</strong>
    @endif
    {{ $slot }}
</div>
```

Utiliser le composant

```blade
{{-- Utilisation simple --}}
<x-alert type="success">
    Projet créé avec succès !
</x-alert>

{{-- Avec titre --}}
<x-alert type="danger" title="Erreur">
```

Exemple : Composant Card

Pour afficher les projets TaskFlow

Créer le composant

Fichier : `resources/views/components/projet-card.blade.php`

BLADE

```
@props(['projet'])

<div class="card">
    <div class="card-header">
        <h3>{{ $projet->nom }}</h3>
        <span class="badge badge-{{ $projet->statut }}">
            {{ $projet->statut }}
        </span>
    </div>
    <div class="card-body">
        <p>{{ $projet->description }}</p>
        <small>{{ $projet->taches_count }} tâche(s)</small>
    </div>
    <div class="card-footer">
        <a href="{{ route('projets.show', $projet) }}">Voir</a>
    </div>
</div>
```

Utilisation dans une liste

BLADE

```
<div class="projets-grid">
    @foreach($projets as $projet)
```

5. Inclusion de Sous-vues

@include et @each

@include - Inclure une vue partielle

BLADE

```
{{-- Inclure le header --}}
@include('partials.header')

{{-- Inclure avec des données --}}
@include('partials.sidebar', ['menu' => $menuItems])

{{-- Inclure si existe --}}
@includeIf('partials.optional')

{{-- Inclure si condition vraie --}}
@includeWhen($user->isAdmin, 'partials.admin-menu')
```

@each - Inclure pour chaque élément

BLADE

```
{{-- Pour chaque projet, inclure la vue partials.projet-item --}}
@each('partials.projet-item', $projets, 'projet')

{{-- Avec vue pour le cas vide --}}
@each('partials.projet-item', $projets, 'projet', 'partials.no-projets')
```

Fichier partiel exemple

`resources/views/partials/projet-item.blade.php`

Liens et Assets

URLs, routes, images, CSS

Générer des URLs

BLADE

```
{{-- URL de base --}}
<a href="{{ url('/projets') }}">Projets</a>

{{-- Route nommée --}}
<a href="{{ route('projets.index') }}">Liste</a>

{{-- Route avec paramètre --}}
<a href="{{ route('projets.show', $projet->id) }}">Voir</a>

{{-- Route avec modèle (Route Model Binding) --}}
<a href="{{ route('projets.show', $projet) }}">Voir</a>
```

Assets (CSS, JS, Images)

BLADE

```
{{-- Fichiers dans public/ --}}
<link href="{{ asset('css/app.css') }}" rel="stylesheet">
<script src="{{ asset('js/app.js') }}"></script>
<img src="{{ asset('images/logo.png') }}" alt="Logo">

{{-- Avec Vite (Laravel 11) --}}
@vite(['resources/css/app.css', 'resources/js/app.js'])
```

Conseil

Formulaires et CSRF

Protection contre les attaques

CSRF - Cross-Site Request Forgery

Laravel protège automatiquement vos formulaires contre les attaques CSRF. Vous devez ajouter `@csrf` dans chaque formulaire.

Formulaire simple

BLADE

```
<form action="{{ route('projets.store') }}" method="POST">
    @csrf

    <label>Nom du projet</label>
    <input type="text" name="nom" required>

    <label>Description</label>
    <textarea name="description"></textarea>

    <button type="submit">Créer</button>
</form>
```

Méthode PUT/DELETE (spoofing)

BLADE

```
{{-- HTML ne supporte que GET et POST --}}
{{-- @method simule PUT, PATCH, DELETE --}}

<form action="{{ route('projets.update', $projet) }}" method="POST">
    @csrf
```

Résumé du Chapitre

{{ $var }}

Afficher une variable (échappé)

@if / @foreach

Structures de contrôle

@extends / @yield

Héritage de layouts

<x-composant />

Composants réutilisables

Directives essentielles

Directive	Usage
`@extends('layout')`	Hériter d'un layout
`@section('nom') ... @endsection`	Définir une section
`@yield('nom')`	Afficher une section
`@include('vue')`	Inclure une sous-vue
`@csrf`	Token de protection CSRF

Prochain chapitre

Chapitre 4 : Bases de données et Migrations

Questions ?

Avez-vous des questions sur Blade ?

Prochaine séance : TP 3

Création des interfaces de TaskFlow avec Blade